



Escola Superior de Tecnologia do Cávado e do Ave

Engenharia Eletrotécnica e de Computadores

Robótica

a20478 | Luís Barrão
a23067 | Alexandre Jesus
a23080 | Paulo Palhares

Trabalho Prático 2 – “Controlo, Navegação e Visão em Robot Móvel Lego *Mindstorms*”

Professor: Pedro Morais

Janeiro de 2026

Resumo

O presente relatório descreve o desenvolvimento de um sistema autónomo de controlo e navegação aplicado a um robô móvel Lego *Mindstorms*, operando num ambiente estruturado e controlado exclusivamente por visão computacional. O projeto focou-se na substituição da odometria e sensorização local clássica por uma arquitetura de perceção global, onde uma câmara em configuração *top-down* fornece os dados necessários para o mapeamento métrico do espaço e a localização em tempo real do veículo. A metodologia adotada integra algoritmos de planeamento de trajetória probabilísticos (*Probabilistic Roadmap* - PRM) com controladores cinemáticos de execução ponto-a-ponto (*Drive2Point*) e de seguimento contínuo (*Pure Pursuit*). Adicionalmente, foi implementado um módulo de reconhecimento semântico baseado em processamento de imagem, permitindo ao robot identificar sinais de trânsito e tomar decisões de navegação autónomas. Os resultados experimentais validaram a eficácia da navegação baseada em visão externa, evidenciando, contudo, a necessidade de compensação mecânica face às limitações de tração do chassis em manobras de rotação estacionária.

Índice

Introdução	5
Contextualização	5
Objetivos do Trabalho	5
Desenvolvimento	6
Configuração do Sistema	6
Descrição do Robô LEGO.....	6
Programa e suas funções	6
GUI (Graphical User Interface).....	6
Parâmetros de câmara.....	7
Parâmetros HSV para threshold	7
Processamento da Imagem	7
Mapeamento por Visão Computacional.....	8
Parâmetros para criação de mapa.....	8
Diagrama de Processamento de Imagem para Mapeamento	8
Criação do mapa	9
Plots	11
Parâmetros do robô.....	11
Cinemáticas do robô	11
Controlo do robô	12
Posição do robô	13
Ordem dos waypoints.....	13
Trajetória (Path).....	13
PRM (<i>Probabilistic Roadmap</i>)	14
Drive2Point	14
Pure Pursuit	15
Reconhecimento e classificação de Sinais	16
Conclusão.....	17
Resultados e discussão	17
Objetivos cumpridos.....	17
Limitações.....	18
Trabalho futuro	18
Melhorias	18
WebGrafia	19

Índice de Figuras

Figura 1 - Robô Lego Mindstorm EV3	6
Figura 2 - Interface Gráfica	6
Figura 3 - Fluxograma de processamento de imagem.....	7
Figura 4 - Fluxograma do cálculo de ratio.....	8
Figura 5 - Fluxograma de obtenção de mapa	9
Figura 6 - Pista com objetos e robô	10
Figura 7 - Fluxograma de aquisição de posição do robô	13
Figura 8 - Fluxograma de definição dos waypoints	14
Figura 9 - Fluxograma de movimentação do robô.....	15
Figura 10 - Fluxograma de tomada de decisão na interpretação do sinal.....	16

Introdução

Contextualização

O desenvolvimento de sistemas robóticos autónomos exige a orquestração precisa entre a capacidade de perceber o meio envolvente e a competência para atuar sobre ele, assim, o presente trabalho visa juntar a visão computacional do trabalho passado com o controlo e navegação de um robô Lego *Mindstorms*, conseguindo mapear e evitar a colisão com obstáculos. O sistema dispõe de deteção em tempo real de sinais de trânsito adicionando uma camada de complexidade extra ao trabalho.

Objetivos do Trabalho

O trabalho consiste em abordar e entender as cinemáticas de um robô no mundo real através da aquisição de uma imagem sobre uma pista, criando um mapa com uma escala real e adquirindo a posição do robô e das bases de diferente cor na pista, fazendo depois o robô movimentar-se entre elas. O objetivo final é fazer com que o robô consiga através das cinemáticas se movimentar de forma a seguir uma ordem de bases, mas considerando as bases como obstáculos. Por fim acrescentar uma parte do trabalho anterior onde percebemos como abordar a aquisição de imagem e formas de morfologia da mesma mais a classificação de sinais. Sinais estes que estarão situados sobre as bases e que pretendemos que o robô com uma outra câmara que estará na parte da frente do robô adquira uma imagem do sinal e conforme o sinal, que o robô realize a ordem/movimento que o sinal lhe envia.

Desenvolvimento

Configuração do Sistema

Descrição do Robô LEGO

Robô montado de forma que a parte da frente fosse a zona onde estão as rodas, o que permite com que o eixo de rotação seja ao centro do eixo das rodas. Com um raio de roda de 2,8 centímetros e uma distância entre rodas de 12 centímetros.

Na parte superior do robô foi necessário criar 2 círculos com áreas diferentes, para ajudar a definir qual a frente do robô e para ajudar na criação da escala do mapa, utilizando a distância entre eles como referência (15 centímetros).



Figura 1 - Robô Lego Mindstorm EV3

Programa e suas funções

GUI (Graphical User Interface)

A Interface Gráfica de Utilizador, também designada por GUI (*Graphical User Interface*), constitui uma camada de abstração visual fundamental que gere a interação entre o operador e o sistema computacional lógico. A sua finalidade principal reside na maximização da interação, convertendo linhas de código e estruturas de dados complexas em elementos gráficos intuitivos, tais como botões, menus e painéis de visualização. No trabalho prático é utilizada para permitir que a interação com o robô e alteração dos parâmetros do robô seja possível sem ter de ser efetuada qualquer alteração ao código fonte em tempo real.

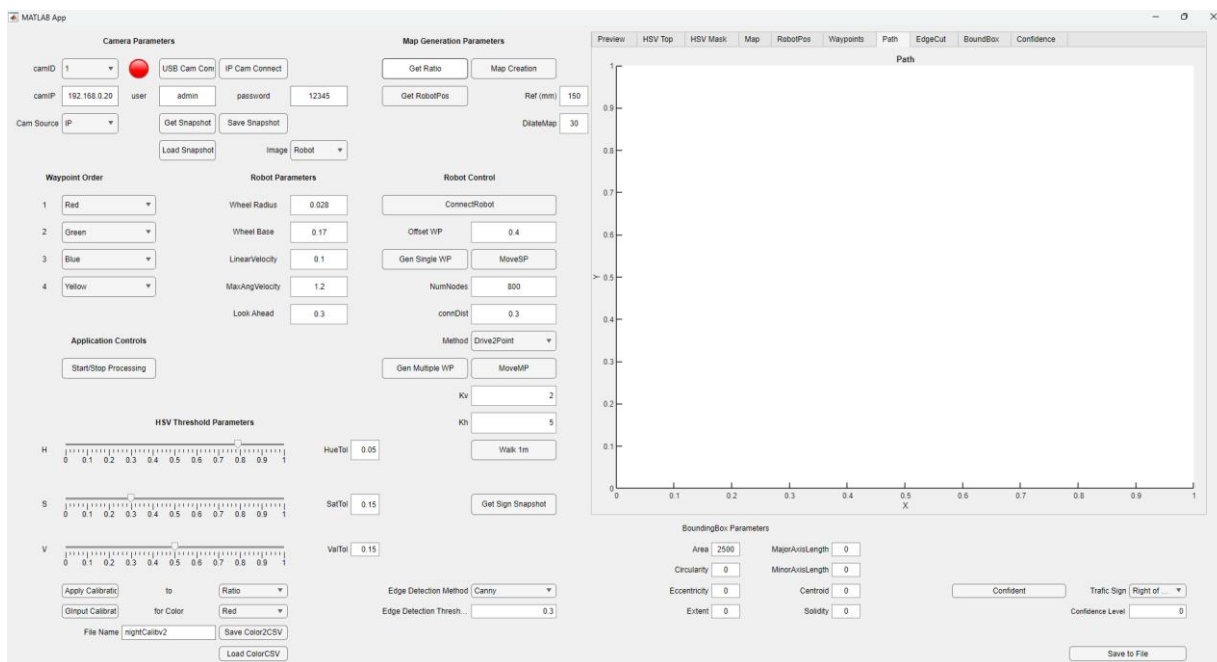


Figura 2 - Interface Gráfica

Parâmetros de câmara

De uma forma geral, a GUI está estruturada tanto para receber uma imagem através de um IP, de uma ligação USB e também para aceder a uma pasta com imagens guardadas que servem como exemplo e para testes de *DEBUG*.

Assim que selecionado o método para obtenção de imagem, apenas é necessário pressionar os botões para qualquer processo que chame pela aquisição da mesma. Há também um botão para guardar a imagem obtida numa pasta predefinida para guardar imagens exemplo.

Parâmetros HSV para threshold

Composto por 3 *sliders* para cada canal do *HSV* com as suas respetivas tolerâncias em campos de texto editáveis.

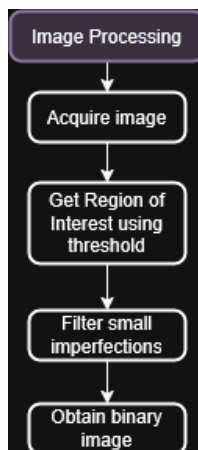
Para utilizar estes *thresholds* é necessário primeiro o tipo e a cor que estamos a trabalhar e para os quais serão enviados estes valores dos *thresholds*. Uma vez concluída a verificação dos *thresholds* é possível clicar no botão que aplica os *thresholds* e, por último, uma vez os *thresholds* aplicados considerou-se útil a criação de um ficheiro onde fosse possível guardar estas variáveis para que apenas fosse necessário fazer os *thresholds* uma única vez. Apenas é necessário dar um nome ao ficheiro e clicar no botão guardar. Quando se utilizar a GUI novamente, apenas é necessário carregar os valores já guardados no(s) ficheiro(s).

Processamento da Imagem

Visto que os fundamentos de processamento de imagem já haviam sido consolidados na etapa anterior, o foco neste trabalho incidiu na simplificação e correção estrutural do código. Para evitar redundâncias. Criou-se uma função genérica de tratamento morfológico gerida por uma estrutura *switch-case*. Esta implementação permite a reutilização de código, onde a chamada de operações de morfologia é feita de forma dinâmica, bastando definir o tipo de ação e as propriedades do *strel* a aplicar.

Assim, no que toca às operações de morfologia, há 2 funções principais:

- A operação de binarização da imagem, que recebe um *array* com os valores de *threshold* HSV usados para filtrar a região de interesse
- A operação de filtragem, que aplica diversos métodos de remoção de pequenos defeitos ou seleção de objetos com base na área para garantir que conseguimos um objeto claro e simples para a definição dos parâmetros de trabalho.



FFigura 3 - Fluxograma de processamento de imagem

Mapeamento por Visão Computacional

O mapeamento baseado em visão computacional é uma técnica que permite a um sistema robótico reconstruir e interpretar a geometria do ambiente através de sensores óticos, em substituição ou complemento aos sensores de distância tradicionais, como LIDAR ou sonar. O princípio fundamental reside na extração de informação métrica a partir de dados visuais, transformando a projeção bidimensional de uma imagem num modelo de representação espacial, frequentemente sob a forma de grelhas de ocupação (*Occupancy Grids*). Para que este mapa seja útil à navegação autónoma, é necessário estabelecer uma correspondência entre o plano da imagem e o plano do mundo físico, um processo que envolve a correção de distorções de perspetiva e a aplicação de uma escala métrica. Desta forma, o sistema consegue distinguir zonas navegáveis de obstáculos, fornecendo uma base estruturada para os algoritmos de localização e planeamento de trajetórias.

Parâmetros para criação de mapa

Existem 3 botões para a concretização da criação do mapa:

- O botão que chama a função permite obter o ratio entre pixéis e milímetros
- O botão que chama a função que cria o mapa binário com a escala real
- O botão que permite obter a posição do robô.

Para além disso existe 2 campos de edição: um campo onde nos é possível definir a medida que utilizamos para obter a escala real do mapa, sendo que o código utiliza a distância entre os 2 centroides dos círculos que estão colocados na parte superior do robô para ter uma medida de referência, e o outro campo onde nos é possível escolher a quantidade de pixéis que o mapa irá dilatar para criar o obstáculo e evitar a colisão do robô com os objetos.

Diagrama de Processamento de Imagem para Mapeamento

Para criação do mapa é necessário entender que a fotografia recebida do telemóvel está em pixéis e é necessário criar um mapa em metros. Ou seja, inicialmente realiza-se um cálculo de *ratio* que permite transformar os pixéis em milímetros e posteriormente transformar os milímetros em metros.

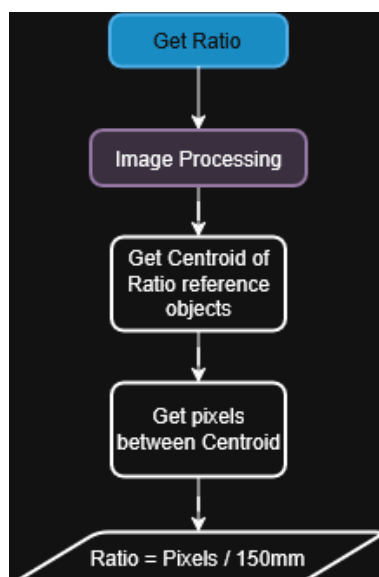


Figura 4 - Fluxograma do cálculo de ratio

O seguinte diagrama explica que através dos dois círculos colocados na parte superior do robô, obtendo os respectivos centroides de ambos os objetos, sabendo que estes se encontram a uma distância de 150 milímetros, utilizamos o valor de pixels sobre essa distância para o cálculo do *ratio* sendo assim possível criar o mapa com a escala real.

$$ratio = abs(\frac{dx}{referenceInMm})$$

Sendo:

abs: Refere que o que se encontra dentro de parênteses tem de ser um valor absoluto.

dx: Diferença entre os centroides dos dois marcadores de calibração.

referenceInMm: Distância real em milímetros.

- **Resultado:** Devolve um fator de escala (pixels por mm) que será usado para converter qualquer coordenada da imagem para metros.

Criação do mapa

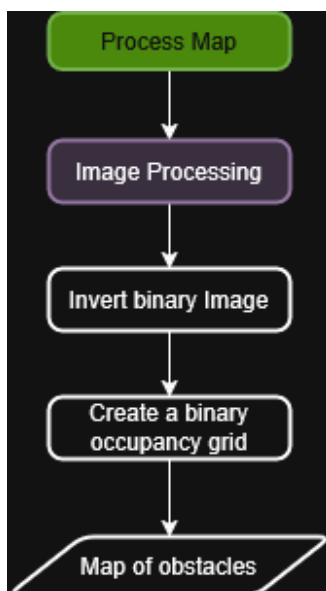


Figura 5 - Fluxograma de obtenção de mapa

Este módulo é responsável pela conversão da informação visual processada numa estrutura de dados métrica para navegação. O processo transforma a matriz binária processada num objeto *binaryOccupancyMap*, fundamental para a orientação do robô.

O procedimento divide-se em três etapas:

- O algoritmo recebe a imagem binária onde os valores lógicos "1" são interpretados como obstáculos e os valores "0" como espaço livre para o robô navegar.
- Converte a escala obtida anteriormente de *pixéis/mm* para *pixéis/metro*.

$$scale_metros = ratio * 1000$$

1. Instância um objeto *binaryOccupancyMap* do MATLAB.

Importância:

Este processo é crucial para o planeamento das trajetórias. Permite que os algoritmos de navegação operem com coordenadas reais, dissociando a lógica de controlo das características intrínsecas da câmara (resolução da imagem).

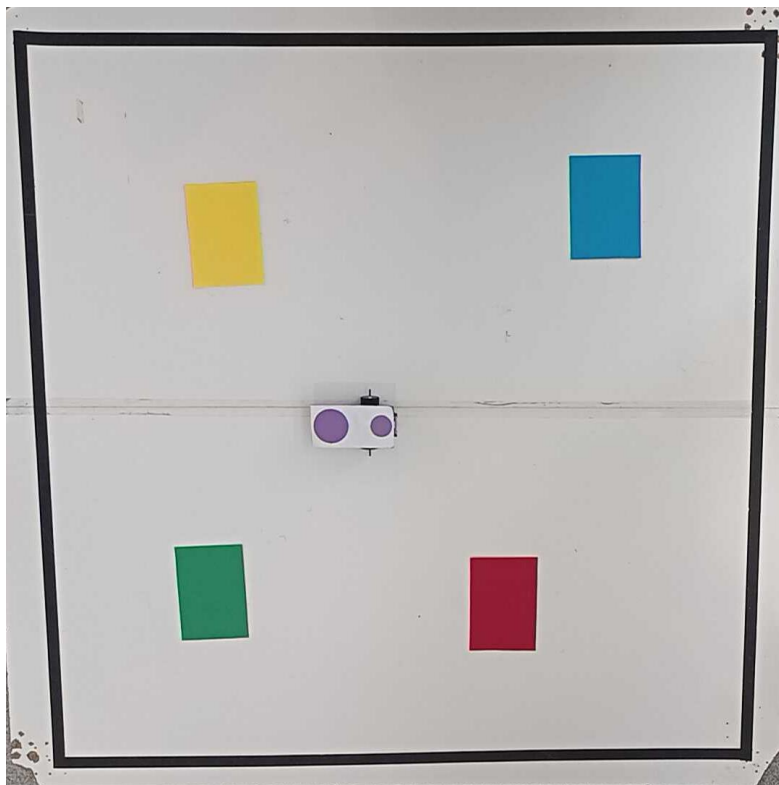


Figura 6 - Pista com objetos e robô

Plots

Considerou-se necessário haver várias janelas na GUI para um *DEBUG* mais facilitado, criou-se então as seguintes:

- **Preview:** Permite ver a imagem original do topo.
- **HSV Top:** A imagem original do topo, mas com os valores de HSV para permitir uma melhor calibração das cores, usando os *sliders* anteriormente mencionados.
- **HSV Mask:** Uma imagem binária criada a partir dos valores adquiridos na imagem HSV e colocados nos *sliders* como *thresholds*, importante e necessário no ajuste dos valores para deteção de *Roi* (*Region of Interest*), confirmando assim que é aquilo que queremos que seja identificado.
- **Map:** Permite ver a criação do mapa binário, sendo possível ver a pista recortada pelos seus limites externos e as suas respetivas bases.
- **RobotPos:** Usada para verificar posição do robô através de um *snapshot* que utiliza a área do *regionprops* e define o *centroid* do círculo mais pequeno, dando a posição do robô, sendo possível mostrar também o ângulo que o robô tem no momento.
- **Waypoints:** Cria 4 *plots* diferentes com a mesma imagem de fundo, onde mostra os *waypoints* que foram criados pela ordem que estava na respetiva seleção feita na GUI.
- **Path:** Indica a trajetória do robô desde o ponto onde assumiu a sua posição até ao próximo ponto que vai percorrer.

Parâmetros do robô

Campos editáveis onde podemos definir o raio da roda presente no robô que está a ser utilizado, a distância entre o centro de ambas as rodas, a sua velocidade linear e velocidade angular, e o ponto para o qual o robô está a olhar quando se movimenta dependendo dos métodos utilizados. Estes parâmetros são essenciais para a movimentação correta do robô.

Cinemáticas do robô

Cinemática Direta (*Forward Kinematics*): A cinemática direta constitui o mecanismo de tradução entre o espaço dos atuadores e o espaço cartesiano, respondendo à necessidade de estimar o movimento do veículo a partir da rotação dos seus motores.

A cinemática direta tem como:

- **Inputs:** Velocidade da roda direita (ω_R) e esquerda (ω_L).
- **Outputs:** Velocidade linear do robot (v) e velocidade angular (ω).

Considerando r como o raio da roda e L como a distância entre as duas rodas, as equações são:

$$v = \frac{r}{2}(\omega_R + \omega_L)$$
$$\omega = \frac{r}{L}(\omega_R - \omega_L)$$

A partir destas velocidades, integrando ao longo do tempo, obtemos a nova posição (x , y ,

θ) do robot no mapa:

$$\begin{aligned}\dot{x} &= v * \cos \theta \\ \dot{y} &= v * \sin \theta \\ \dot{\theta} &= \omega\end{aligned}$$

Cinemática Inversa (*Inverse Kinematics*): Por sua vez, a cinemática inversa opera a conversão oposta, traduzindo as intenções de movimento do espaço cartesiano para o espaço dos atuadores. É essencial para determinar como os motores devem rodar para atingir uma velocidade desejada.

A cinemática inversa tem como:

- **Inputs:** Velocidade linear desejada (v) e angular desejada (ω).
- **Outputs:** Velocidades necessárias para a roda direita (ω_R) e esquerda (ω_L).

As equações obtêm-se isolando ω_R e ω_L das fórmulas anteriores:

$$\omega_R = \frac{v + \frac{L}{2}\omega}{r}$$

$$\omega_L = \frac{v - \frac{L}{2}\omega}{r}$$

Controlo do robô

Fazendo a conexão com o robô, após receber um *beep* de confirmação bem sucedida, é possível escolher o método que o robô vai utilizar para se mover: o método *Drive2Point* ou *Pure Pursuit*.

Deste modo, pode ser gerado o trajeto com os *waypoints* que foram obtidos anteriormente, e através do *PRM (Probabilistic Roadmap)* em conjunto com o mapa criado, é possível gerar um trajeto livre de obstáculos. No PRM, é o utilizador define a quantidade pontos presentes no mapa e a distância de conexão entre os mesmos, este fará o caminho mais pequeno percorrendo os vários pontos aleatoriamente espalhados para chegar desde a sua posição inicial à posição final.

Posição do robô

A determinação da posição do robô, baseia-se na identificação de dois círculos colocados no topo do robô: um círculo menor, que indica a frente, e um círculo maior, que indica a traseira.

Com a binarização da imagem no espaço de cor HSV e utilizando os filtros morfológicos é possível isolar os objetos de interesse. De seguida, o algoritmo extrai os centroides e as áreas dos *blobs* detetados, ordenando-os por tamanho para distinguir qual corresponde à frente e qual à traseira.

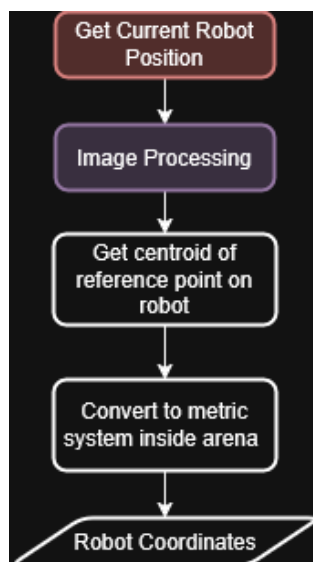


Figura 7 - Fluxograma de aquisição de posição do robô

Ordem dos waypoints

É possível seleccionar a ordem pelo qual o robô vai navegar pelas bases em qualquer combinação possível entre estas, ficando assim possível ao utilizador definir como quer que o robô realize o seu percurso utilizando o menu seletor na GUI.

Trajectoria (Path)

Para obtenção da trajetória que o robô deve percorrer, o programa interpreta a ordem pela qual estão organizadas as bases na secção *Waypoint Order*. Deste modo, é possível fazer-se o ajuste de qual a trajetória que o robô deverá fazer sem que seja necessário alterar fisicamente a posição das bases na pista.

Para obtenção dos pontos que o robô deve seguir, pela ordem definida, é aplicado o processo de binarização para aquisição da região de interesse utilizando os valores do *array* HSV ligado num par *key-value* com a *string* do menu de seleção, sendo posteriormente adquirida a Centroid da base para usar como referência. A partir da Centroid, são estabelecidos mais 4 waypoints com offsets de distância, permitindo que a área do robô seja tida em consideração para a definição do trajeto. Desta forma, é gerado um array de pontos que, em conjunto com os pontos gerados pelo mecanismo de PRM, permite ao robô deslocar-se para o local pretendido evitando os obstáculos definidos na criação do mapa.

Dada a forma como a imagem é obtida, há a necessidade de inversão do valor da coordenada em *Y* para que o robô se desloque no sentido correto. Deste modo, é feito um cálculo:

$$Y \text{ (Coordinate value)} = \text{image size in Y} - \text{centroid Y value}$$

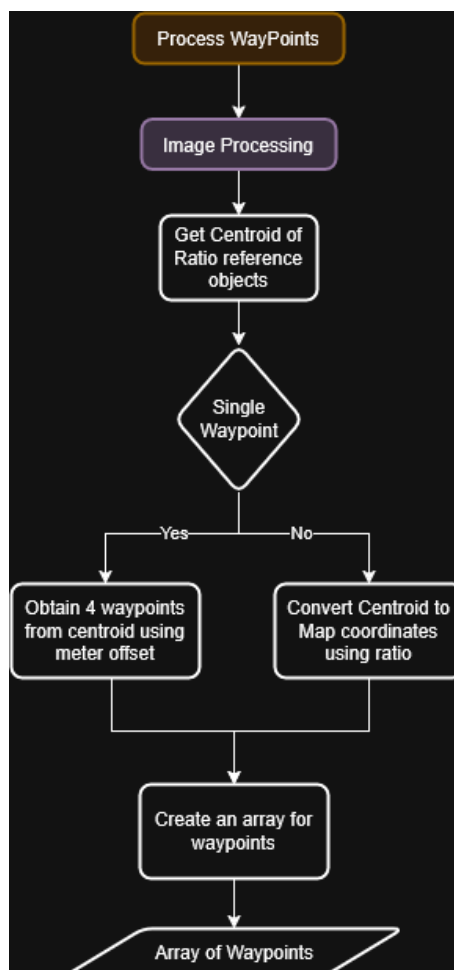


Figura 8 - Fluxograma de definição dos waypoints

PRM (*Probabilistic Roadmap*)

Para garantir uma navegação segura na pista com os obstáculos (bases), o sistema recorre ao algoritmo PRM (*Probabilistic Roadmap*). Este método de planeamento constrói um grafo de conectividade sobre o espaço livre do mapa de ocupação (*Occupancy Grid*), distribuindo *nodes* na grelha e validando se as ligações (arestas) entre eles interseccionam zonas proibidas. Uma vez estabelecida esta rede de caminhos, o algoritmo determina a rota mais curta entre a posição atual do robô e a base destino definida pelo *array* de *waypoints* criado anteriormente, gerando uma lista sequencial de coordenadas intermédias (*waypoints*). Esta abordagem permite contornar obstáculos, decompondo a trajetória global em segmentos lineares seguros que servem de referência para os controladores de movimento.

Drive2Point

O método *Drive2Point* é responsável por levar o robô da sua posição atual até um ponto exato no *array* de objetivos, percorrendo sequencialmente a lista de nós gerada pelo algoritmo PRM, repetindo este ciclo até terminar os *waypoints* definidos no *array*. Para garantir que o movimento é preciso, foi implementada uma lógica de duas fases baseada no erro de orientação. Primeiro, verifica se o robô está virado para o alvo: caso o desvio seja significativo (superior a 15º), o veículo roda sobre si mesmo sem avançar (*turn-in-place*) até corrigir o alinhamento. Assim que a orientação está correta, o robô inicia o deslocamento em frente, ajustando a velocidade linear e angular simultaneamente para se manter na rota direta. Este ciclo repete-se constantemente até que o robô esteja a menos de 30 milímetros do objetivo final.

Pure Pursuit

O método *Pure Pursuit* é uma abordagem para suavizar a execução da trajetória calculada pelo PRM. Ao contrário do método ponto-a-ponto, este algoritmo processa a lista completa de *waypoints* do PRM como um caminho contínuo. Utilizando o objeto *controllerPurePursuit*, o sistema calcula dinamicamente a curvatura necessária para perseguir um ponto virtual (*lookahead point*) que se desloca sobre os segmentos definidos pelo PRM, a uma distância fixa à frente do veículo. Esta técnica permite ao robô "cortar" suavemente as esquinas dos *waypoints*, gerando um movimento fluído e contínuo sem paragens intermédias.

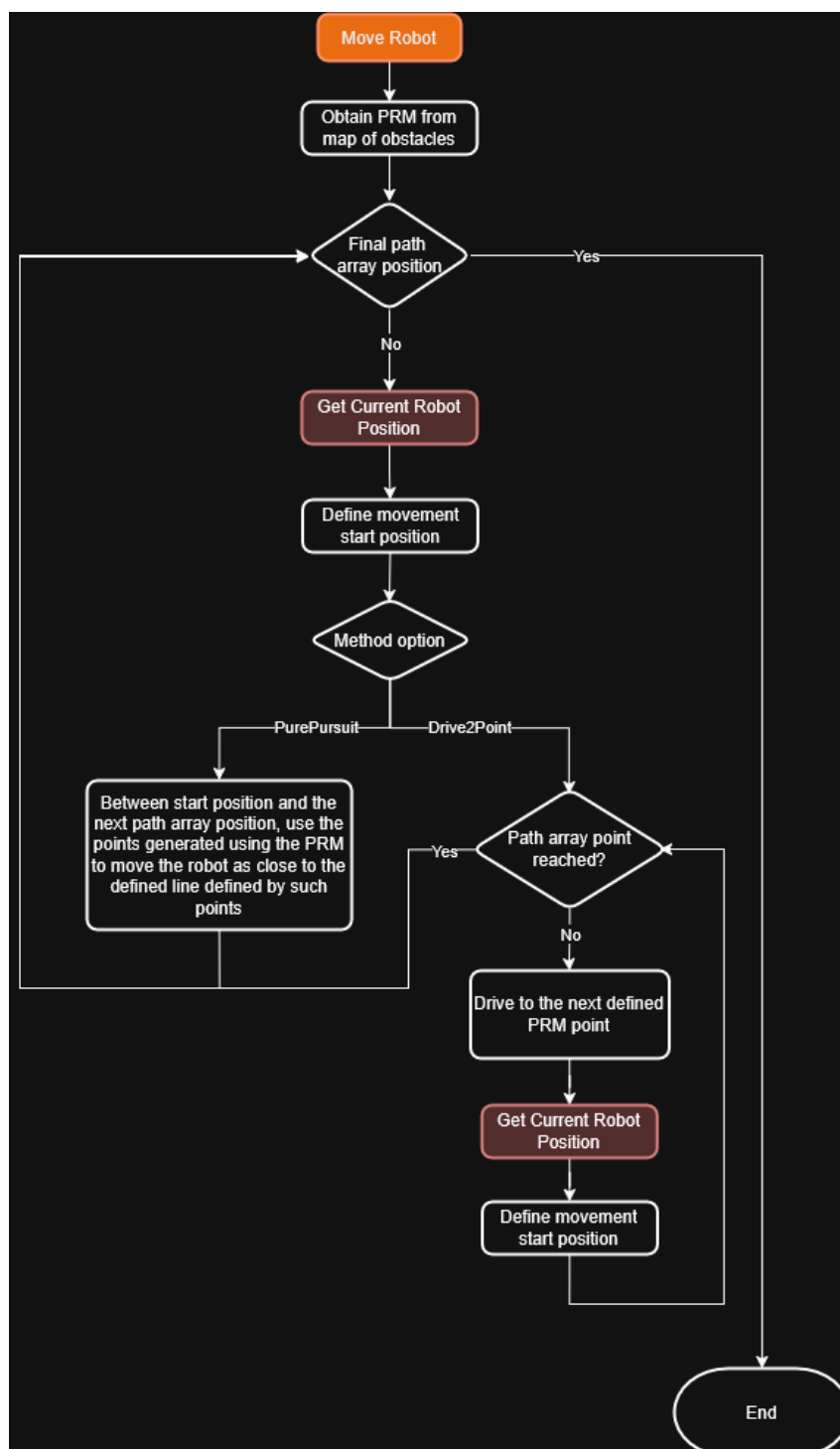


Figura 9 - Fluxograma de movimentação do robô

Reconhecimento e classificação de Sinais

O processo de reconhecimento do sinal começa com a conexão à câmara do robô, captura da imagem e a procura imediata pela cor azul, o que permite isolar o sinal de trânsito do resto do ambiente através de uma binarização. Uma vez localizado o sinal, a imagem é recortada e convertida para preto e branco. Para garantir a leitura correta é aplicada uma máscara circular no centro da imagem, que serve para "apagar" o aro branco do sinal e deixar apenas a seta visível. Por fim, a decisão é tomada comparando a posição da ponta da seta com o centro da imagem: se a ponta estiver mais à esquerda que o centro, o sistema assume a instrução para virar à esquerda; caso contrário, segue para a direita.

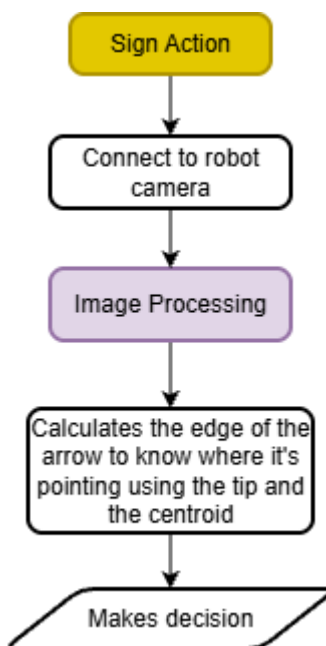


Figura 10 - Fluxograma de tomada de decisão na interpretação do sinal

Conclusão

O presente trabalho permitiu consolidar os conceitos fundamentais de robótica móvel, permitindo o desenvolvimento de um sistema de navegação e controlo autónomo. A integração de uma arquitetura de visão global com algoritmos de planeamento probabilístico (PRM) e controladores cinemáticos demonstrou ser uma metodologia eficaz para assegurar a navegação autónoma e precisa do veículo num cenário controlado. Ao longo das secções seguintes, apresenta-se uma reflexão crítica sobre os resultados obtidos, o grau de cumprimento dos objetivos propostos e as condicionantes encontradas durante o desenvolvimento do projeto.

Resultados e discussão

Os testes efetuados em ambiente real demonstraram que a estratégia de controlo ponto-a-ponto (*Drive2Point*) obteve resultados consistentes e precisos no seguimento da trajetória gerada pelo PRM. Contudo, durante a fase de validação da leitura de sinais, identificou-se uma limitação: após a chegada à base, o robô necessitava de rodar para alinhar a câmara nele colocada com o centroide do sinal. Verificou-se que a rotação real do robô era frequentemente inferior à necessária, devido a fatores externos como a acumulação de sujidade nas rodas, irregularidades da pista e a variabilidade do atrito no piso.

Para mitigar este erro, foi necessário calibrar os ganhos de rotação e implementar lógicas de compensação. Embora estas correções na resposta mecânica pudessem, teoricamente, beneficiar também o desempenho do algoritmo *Pure Pursuit* (que depende fortemente da precisão da odometria em curvas), não foi possível reavaliar este método após os ajustes. Assim, o *Drive2Point* prevaleceu como a solução mais fiável para a demonstração final, garantindo o alinhamento correto para o reconhecimento dos sinais de trânsito.

Objetivos cumpridos

Apesar dos desafios encontrados, a maioria dos objetivos pedagógicos e funcionais foram atingidos com sucesso:

- **Implementação Cinemática:** O modelo diferencial direto e inverso foi corretamente transposto para código, permitindo o controlo de velocidade linear e angular.
- **Mapeamento e Localização:** O sistema de visão global (*top-down*) provou ser eficaz na criação do mapa de ocupação e na localização em “tempo real” do robô, substituindo a necessidade de sensores de localização internos.
- **Planeamento de Trajetórias:** A integração do algoritmo PRM permitiu gerar trajetos livres de colisão, adaptando-se dinamicamente à posição dos obstáculos detetados.
- **Navegação Autónoma:** O robô demonstrou capacidade para visitar a sequência de bases definida e imobilizar-se corretamente.
- **Visão Computacional:** O módulo de reconhecimento de sinais foi capaz de identificar a sinalização e influenciar a decisão de navegação (escolha do próximo *waypoint*).

Limitações

O desenvolvimento do projeto foi condicionado pelos seguintes fatores que impactaram a otimização final do sistema:

- **Acesso à pista:** A existência de apenas uma pista física para toda a turma limitou o tempo disponível para testes reais.
- **Setup experimental:** A montagem do suporte da câmara e a garantia de condições de iluminação ideais consumiram tempo significativo em cada teste, reduzindo a janela útil de trabalho.
- **Upgrade da GUI:** A necessidade de reestruturar a Interface Gráfica (GUI) e os algoritmos de processamento de imagem (aquisição e morfologia) desviou recursos e tempo que idealmente seriam alocados ao refinamento da cinemática e ao teste exaustivo do controlador *Pure Pursuit*.

Trabalho futuro

Melhorias

Como perspectivas de evolução do projeto/sistema, destaca-se a necessidade de ajustar os parâmetros dos controladores cinemáticos (*Drive2Point* e *Pure Pursuit*), de forma a incorporar modelos de compensação para variáveis físicas como o atrito do piso e a acumulação de sujidade nas rodas, fatores que demonstraram introduzir erros sistemáticos na odometria e na rotação.

Ao nível do *software*, impõe-se uma reestruturação da Interface Gráfica (GUI) e do código-fonte, visando a remoção de funcionalidades herdadas de projetos anteriores, o que resultaria numa aplicação mais leve e intuitiva.

Por fim, propõe-se a transição para um paradigma de navegação mais realista, operando exclusivamente com a câmara inserida no robô num ambiente com obstáculos tridimensionais. Esta abordagem permitiria ao robô estimar a profundidade e a distância aos objetos através da análise da sua área aparente (utilizando *regionprops*), simulando com maior fidelidade os desafios de uma condução real.

WebGrafia

<https://ww2.mathworks.cn/help/images/morphological-filtering.html> - (Última verificação: 29-01-2026)

<https://ww2.mathworks.cn/help/images/ref/regionprops.html> - (Última verificação: 29-01-2026)

<https://ww2.mathworks.cn/help/nav/ref/binaryoccupancymap.html> - (Última verificação: 29-01-2026)

<https://ww2.mathworks.cn/en/discovery/path-planning.html> - (Última verificação: 29-01-2026)

<https://ww2.mathworks.cn/help/nav/ref/controllerpurepursuit-system-object.html> - (Última verificação: 29-01-2026)

<https://ww2.mathworks.cn/help/robotics/ref/mobilerobotprm.html> - (Última verificação: 29-01-2026)

<https://ww2.mathworks.cn/help/nav/ug/pure-pursuit-controller.html> - (Última verificação: 29-01-2026)